SpiralTrain
developer courses



Python Libraries and Tools

Optional Topics

Choosing Topics

- The optional modules are selected with the trainer to suit the group.
- Command-line programs and subprocesses support scripting and automation.
- The modern toolbelt covers progress displays, formatted output and HTTP.
- Web scraping and SQLAlchemy cover collecting and storing records.
- NumPy and pandas cover arrays, tables and reports.
- GUI toolkits introduce desktop windows.
- Inheritance previews Advanced Python and can be skipped.
- The supplied data and database seed let later modules run independently.

- ▶ **Inheritance**
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- NumPy
- Pandas
- GUI Toolkits

APPENDIX

- Running Other Programs

- Inheritance
- Duplicated Code
- Cost of Duplication
- Base Classes and Subclasses
- Inherited Members
- Constructor Chaining
- Overriding Methods
- Extending with super
- isinstance and issubclass
- str and repr
- Polymorphism
- Abstract Base Classes
- Enforcing the Contract
- Composition
- Scraper Toolkit
- Advanced Python Topics

SpiralTrain developer courses

Inheritance

- A subclass inherits behaviour from a base class
- Related objects can share an interface while supplying different methods
- `Mage`, `Warrior` and `Priest` specialise `Character`
- Health, mana and rage values here are simplified teaching data

SpiralTrain developer courses

Duplicated Code

- Both classes initialise a name and health
- A change to shared behaviour requires editing both implementations
- A base class can hold that common behaviour

```
class Mage:
    def __init__(self, name: str) -> None:
        self.name: str = name
        self.health: int = 100
        self.mana: int = 100


class Warrior:
    def __init__(self, name: str) -> None:
        self.name: str = name
        self.health: int = 100
        self.rage: int = 0
```

demo01
duplication

SpiralTrain developer courses

Cost of Duplication

- Repeated logic can diverge when one copy changes
- Inheritance puts shared behaviour in one implementation
- Subclasses retain their specific state and methods

SpiralTrain developer courses

Base Classes and Subclasses

- A subclass names its base in parentheses
- The base implementation remains available through the subclass
- An empty subclass can inherit the initialiser and methods unchanged

```
class Character:
    def __init__(self, name: str) -> None:
        self.name: str = name
        self.health: int = 100

    def attack(self) -> str:
        return f"{self.name} attacks"

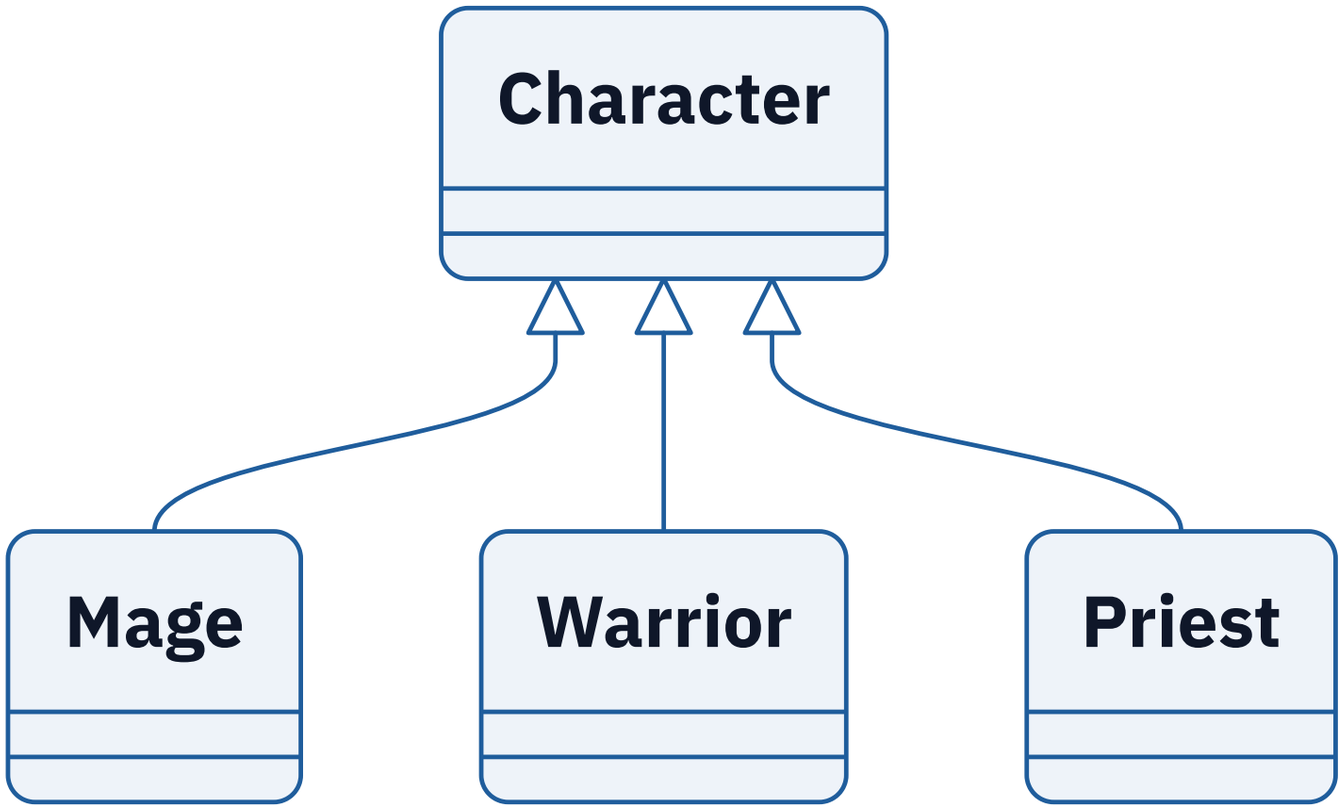

class Mage(Character):
    pass
```

demo02
subclass

SpiralTrain developer courses

Inherited Members

- Inherited methods are found through the class hierarchy
- The base initialiser creates attributes on the actual subclass instance
- The object remains an instance of both its class and its base



SpiralTrain developer courses

Constructor Chaining

- A subclass initialiser can call `super().__init__`
- The base initialiser stores shared attributes on the same object
- The subclass then adds its own attributes

```
class Mage(Character):
    def __init__(self, name: str, mana: int = 100) -> None:
        super().__init__(name)
        self.mana: int = mana
```

demo03
super_init

Overriding Methods

- Defining the same method name in a subclass overrides the base method
- Method lookup starts at the instance's class and follows its method resolution order
- `Mage.attack` is found before `Character.attack`

```
def attack(self) -> str:
        return f"{self.name} casts Frostbolt"

jaina = Mage("Jaina")
print(jaina.attack())
print(Character.attack(jaina))
```

demo04
overriding

SpiralTrain developer courses

Extending with super

- `super()` delegates according to the method resolution order
- An initialiser can extend shared setup without copying it
- Both initialisers receive the same `self` object

```
class Mage(Character):
    def __init__(self, name: str, mana: int = 100) -> None:
        super().__init__(name)
        self.mana: int = mana
```

SpiralTrain developer courses

isinstance and issubclass

- `isinstance` includes inherited relationships
- `issubclass` asks about classes rather than instances
- Exact `type(...)` equality excludes subclass instances

```
jaina = Mage("Jaina")
print(isinstance(jaina, Mage))
print(isinstance(jaina, Character))
print(issubclass(Mage, Character))
print(type(jaina) is Character)
```

SpiralTrain developer courses

str and repr

- `__str__` supplies a readable display value
- `__repr__` supplies a debugging representation
- Containers display their elements with `repr`

```
def __str__(self) -> str:
        return self.name

    def __repr__(self) -> str:
        return f"{type(self).__name__}(name={self.name!r})"
```

demo06
str_and_repr

SpiralTrain developer courses

Polymorphism

- One loop can call the same method on different subclass instances
- Each object supplies its own implementation
- The caller does not need a branch for each class

```
raid: list[Character] = [Mage("Jaina"), Warrior("Varian"), Priest("Anduin")]
for member in raid:
    print(member.attack())
```

demo07
polymorphism

SpiralTrain developer courses

Abstract Base Classes

- `ABC` and `abstractmethod` declare a required method
- A concrete subclass must implement every abstract method
- Abstract classes can also provide ordinary methods and initialisers

```
class Ability(ABC):
    @abstractmethod
    def cast(self) -> str:
        """Return the ability's action."""


class Frostbolt(Ability):
    def cast(self) -> str:
        return "Frostbolt deals frost damage"
```

demo08
abstract_contract

SpiralTrain developer courses

Enforcing the Contract

- An incomplete subclass fails when it is instantiated
- The error names the missing abstract method
- A concrete subclass remains an ordinary Python class

```
try:
    UnfinishedAbility()
except TypeError as error:
    print(f"TypeError: {error}")
```

SpiralTrain developer courses

Composition

- A character has a mount and an inventory
- Composition stores collaborating objects in attributes
- Inheritance describes a kind of object; composition describes what it holds

```
class Character:
    def __init__(self, name: str, mount: Mount) -> None:
        self.name: str = name
        self.mount: Mount = mount
        self.inventory: Inventory = Inventory()
```

demo09
composition

SpiralTrain developer courses

Scraper Toolkit

- An abstract scraper defines `parse` and shares collection behaviour
- Books and quotes supply different parsers behind the same interface
- This applied example uses local records and makes no network requests
- The scraping module applies the same separation to HTML

demo10
scraper_toolkit

SpiralTrain developer courses

Advanced Python Topics

- **Advanced Python extends this introduction :**
  - Multiple inheritance and mixins
  - Protocols, descriptors and metaclasses
- **These mechanisms are not required for the remaining libraries modules**

SpiralTrain developer courses

Summary : Inheritance

- Base classes share state and behaviour with subclasses
- Overrides supply specialised implementations
- `super` extends inherited behaviour on the same object
- Polymorphic calls use the implementation supplied by each object
- Abstract methods enforce a contract; composition stores collaborators

exercise01
Inheritance

- Inheritance
- **Command-line Programs**
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- NumPy
- Pandas
- GUI Toolkits

APPENDIX

- Running Other Programs

- Command-line Programs
- sys.argv
- Manual Argument Parsing
- ArgumentParser
- Options and Flags
- Mutually Exclusive Options
- Generated Help and Errors
- Subcommands
- Path Arguments
- FileType
- main and the Exit Status
- Standard Output and Standard Error
- Inline Script Metadata
- Executable Scripts
- Command-line Checklist

SpiralTrain developer courses

Command-line Programs

- A command-line tool accepts arguments and options
- Standard output carries results; standard error carries diagnostics
- Exit statuses let the shell distinguish success, matches and errors
- A playlist report can share one interface across scripts and pipelines

SpiralTrain developer courses

sys.argv

- `sys.argv[0]` is the script name
- The remaining entries are strings supplied by the caller
- Parsing and validation are separate from receiving those strings

```
arguments = sys.argv[1:] or ["tracks.csv", "--genre", "Rock", "--limit", "3"]
print(f"program: {Path(sys.argv[0]).name}")
for position, value in enumerate(arguments, start=1):
    print(f"argv[{position}] = {value!r}")

line = "2026-10-12 08:00:00 INFO PLAY Imagine"
print(line.split(maxsplit=4))
```

demo01
sys_argv

SpiralTrain developer courses

Manual Argument Parsing

- Manual parsing must recognise every option and its values
- Missing values, invalid conversions and unknown options need error handling
- `argparse` supplies these checks and generated help

```
elif argv[index] == "--limit":
            index += 1
            limit = int(argv[index])
```

demo02
argv_by_hand

SpiralTrain developer courses

ArgumentParser

- A positional argument is required unless configured otherwise
- `type=int` converts its text before the program uses it
- A duration in seconds can be formatted as minutes and seconds

```
def format_duration(seconds: int) -> str:
    return f"{seconds // 60}:{seconds % 60:02d}"

parser = argparse.ArgumentParser(description="Format a track duration.")
parser.add_argument("seconds", type=int, help="duration in seconds")
args = parser.parse_args(sys.argv[1:] or ["183"])
print(format_duration(args.seconds))
```

demo03
first_parser

SpiralTrain developer courses

Options and Flags

- Dashed names define optional arguments
- `default` supplies a value when an option is absent
- `store_true` creates a flag; `count` counts repeated flags
- `choices` rejects unsupported values

```
parser.add_argument("file", type=Path, nargs="?", default=DATA / "tracks.csv")
parser.add_argument("--genre", choices=["Rock", "Pop", "Soul", "Country", "Electronic"])
parser.add_argument("--limit", type=int, default=5)
parser.add_argument("-v", "--verbose", action="count", default=0)
parser.add_argument("--dry-run", action="store_true")
```

demo04
options_and_flags

SpiralTrain developer courses

Mutually Exclusive Options

- A mutually exclusive group accepts at most one member
- Supplying both duration-order flags is a usage error
- Parsing rejects the conflict with exit status 2

```
order = parser.add_mutually_exclusive_group()
order.add_argument("--longest", action="store_true")
order.add_argument("--shortest", action="store_true")
```

demo05
mutually_exclusive

SpiralTrain developer courses

Generated Help and Errors

- `--help` describes declared arguments and options
- A missing required value or invalid conversion writes a usage error to stderr
- Help exits with status 0; usage errors exit with status 2

```
parser = argparse.ArgumentParser(prog="duration", description="Format seconds.")
parser.add_argument("seconds", type=int)
```

demo06
help_and_errors

SpiralTrain developer courses

Subcommands

- `add_subparsers` registers verbs such as `list`, `stats` and `top`
- Each subparser owns its arguments
- `set_defaults(handler=...)` attaches the implementation function

```
verbs = parser.add_subparsers(dest="command", required=True)
    listing = verbs.add_parser("list", help="list tracks")
    listing.add_argument("--genre")
    listing.add_argument("--limit", type=int, default=5)
    listing.set_defaults(handler=cmd_list)
```

demo07
subcommands

SpiralTrain developer courses

Path Arguments

- `type=Path` converts an argument into a path object
- Conversion does not prove that a file exists or is readable
- Open files after parsing, inside a context manager

```
parser = argparse.ArgumentParser()
parser.add_argument("file", type=Path, nargs="?", default=DATA / "tracks.csv")
args = parser.parse_args()
if not args.file.is_file():
    print(f"Missing file: {args.file}", file=sys.stderr)
    raise SystemExit(2)
```

demo08
paths

SpiralTrain developer courses

FileType

- `argparse.FileType` opens a file while parsing
- It is pending deprecation in Python 3.14
- A later parsing failure can leave the file open
- A `Path` argument followed by `with path.open(...)` gives control over its lifetime

```
with args.source.open(encoding="utf-8") as stream:
    print(f"opened {args.source.name}, first line: {stream.readline().strip()}")
```

demo09
filetype

SpiralTrain developer courses

main and the Exit Status

- `raise SystemExit(main(...))` forwards a result to the shell
- Status 0 means success; nonzero statuses describe other outcomes
- This playlist tool returns 2 when its input cannot be read

```
try:
        rows = load(args.file)
    except OSError as error:
        print(f"playlist: {error}", file=sys.stderr)
        return 2
```

demo10
main_exit_codes

SpiralTrain developer courses

Standard Output and Standard Error

- Stdout carries results that another program can read
- Stderr carries diagnostics without mixing them into the data
- `BrokenPipeError` can occur when a downstream program stops reading

```
def emit(path: Path) -> None:
    print(f"Reading {path.name}", file=sys.stderr)
    for row in load(path):
        print(row["title"])
    print("Finished", file=sys.stderr)
```

demo11
stderr_and_pipes

SpiralTrain developer courses

Inline Script Metadata

- PEP 723 records script dependencies in a comment block
- `uv run script.py` reads the metadata and prepares an environment
- The script can remain one file

```
# /// script
# requires-python = ">=3.12"
# dependencies = ["rich>=13.9"]
# ///
```

demo12
pep723_script

SpiralTrain developer courses

Executable Scripts

- A shebang selects the interpreter on systems that support it
- A script still needs executable permission on Linux and macOS
- On Windows, invoke the script through `uv run`

```
#!/usr/bin/env -S uv run --script
```

SpiralTrain developer courses

Command-line Checklist

- Provide generated help and validate arguments
- Accept an input path or a documented stdin marker
- Keep diagnostic messages on stderr
- Use exit statuses consistently; handle missing files and broken pipes

demo13
unix_citizen

SpiralTrain developer courses

Summary : Command-line Programs

- Read arguments with `argparse`
- Declare positionals, options, exclusive flags and subcommands
- Convert paths while parsing; open files afterwards
- Separate stdout, stderr and exit status
- PEP 723 can declare dependencies beside the script

- Inheritance
- Command-line Programs
- **Modern Toolbelt**
- Web Scraping
- SQLAlchemy
- NumPy
- Pandas
- GUI Toolkits

APPENDIX

- Running Other Programs

- Useful Libraries
- tqdm Progress Bars
- Nested Bars
- Manual Bar Updates
- tqdm.write
- rich Console
- rich Tables
- rich Progress
- rich Tracebacks
- rich inspect
- rich Logging Handler
- HTTP with httpx
- requests and httpx
- typer
- typer Trade-offs
- Finding Libraries
- Judging a Library

SpiralTrain developer courses

Useful Libraries

- `tqdm` adds progress bars to loops
- `rich` renders tables, progress, object details and tracebacks
- `httpx` supplies an HTTP client and an offline mock transport
- `typer` builds command-line interfaces from annotated functions

SpiralTrain developer courses

tqdm Progress Bars

- `tqdm` wraps an iterable without changing its values
- `desc` labels the work; `unit` labels each item
- Pass `total=` when the iterable has no known length

```
for _track in tqdm(tracks, desc="Converting", unit="track"):
    time.sleep(0.01)  # Simulated conversion; no audio files are read.
print(f"Converted {len(tracks)} track records")
rows = (track for track in tracks)
for _track in tqdm(rows, total=len(tracks), desc="From a generator"):
    pass
```

demo01
tqdm_basics

SpiralTrain developer courses

Nested Bars

- An outer bar can count albums and an inner bar tracks
- `leave=False` removes each completed inner bar

```
albums = {"Imagine": ["Imagine", "Jealous Guy"],
          "Thriller": ["Thriller", "Billie Jean"]}
for album, titles in tqdm(albums.items(), desc="Albums"):
    for _title in tqdm(titles, desc=album, leave=False):
        time.sleep(0.02)
```

demo02
tqdm_nested

SpiralTrain developer courses

Manual Bar Updates

- Create a bar with a known total, then call `update`
- A context manager closes the bar when the work finishes
- `set_postfix` displays additional progress values

```
with tqdm(total=len(tracks), desc="Converting", unit="track") as bar:
    for track in tracks:
        time.sleep(0.01)
        bar.update(1)
        bar.set_postfix(title=track["title"][:12])
```

demo03
tqdm_manual

SpiralTrain developer courses

tqdm.write

- `tqdm.write` prints a message while preserving the progress display
- Ordinary terminal writes can interrupt a running bar

```
for track in tqdm(tracks, desc="Checking"):
    if not track["last_played"]:
        tqdm.write(f"Never played: {track['title']}")
```

demo04
tqdm_write

SpiralTrain developer courses

rich Console

- `Console.print` handles markup and Python objects
- Markup uses bracketed style names and `[/]` to close a style
- `Console(stderr=True)` sends diagnostics to standard error

```
console = Console()
console.rule("Playlist")
console.print("[bold]Now playing [cyan]Imagine[/][/] ")
console.print({"title": "Imagine", "duration_s": 183})
```

demo05
rich_console

SpiralTrain developer courses

rich Tables

- Declare columns, then add rows
- Column options control alignment and wrapping
- Row styles and footers can distinguish and summarise entries

```
table = Table(title="Top tracks", row_styles=["", "dim"])
table.add_column("Title", style="cyan")
table.add_column("Plays", justify="right")
for track in sorted(tracks, key=lambda row: int(row["plays"]), reverse=True)[:5]:
    table.add_row(track["title"], track["plays"])
table.show_footer = True
table.columns[0].footer = "Shown"
table.columns[1].footer = "5"
Console().print(table)
```

demo06
rich_table

SpiralTrain developer courses

rich Progress

- `track` wraps an iterable with a progress display
- `Progress` supports several tasks and explicit updates
- Its console keeps messages aligned with progress bars

```
for _song in track(tracks[:3], description="Converting"):
    time.sleep(0.02)
with Progress() as progress:
    task = progress.add_task("Preparing playlist", total=3)
    for _ in range(3):
        time.sleep(0.02)
        progress.update(task, advance=1)
```

demo07
rich_progress

SpiralTrain developer courses

rich Tracebacks

- `install(show_locals=True)` replaces the uncaught-exception display
- `console.print_exception` renders an exception inside an `except` block
- Locals help identify the input value that failed

```
install(show_locals=True)
row = {"title": "Imported track", "duration_s": "unknown"}
try:
    duration = int(row["duration_s"])
except ValueError:
    Console().print_exception(show_locals=True, max_frames=2)
```

demo08
rich_traceback

SpiralTrain developer courses

rich inspect

- `inspect` displays an object's attributes and values
- `methods=True` adds method signatures
- `help=True` includes documentation

```
class Track:
    def __init__(self, title: str, duration_s: int) -> None:
        self.title: str = title
        self.duration_s: int = duration_s

    def minutes(self) -> float:
        """Return the duration in minutes."""
        return self.duration_s / 60

song = Track("Imagine", 183)
inspect(song, methods=True, help=True)
```

demo09
rich_inspect

SpiralTrain developer courses

rich Logging Handler

- `RichHandler` renders messages from the standard `logging` API
- A file handler can also keep a plain log
- `log.exception` includes a traceback for a caught exception

```
logging.basicConfig(level="INFO", format="%(message)s", handlers=[
    RichHandler(rich_tracebacks=True),
    logging.FileHandler(DATA / "player.log", encoding="utf-8"),
])
log = logging.getLogger("player")
log.info(f"PLAY {'Imagine'}")
log.warning(f"Missing local file: {'Yesterday.flac'}")
try:
    int("unknown")
except ValueError:
    log.exception("Invalid imported duration")
```

demo10
rich_logging

SpiralTrain developer courses

HTTP with httpx

- `Client` sends requests and reuses connections
- `MockTransport` supplies responses without a network request
- `raise_for_status` turns error responses into exceptions

```
def example_response(request: httpx.Request) -> httpx.Response:
    return httpx.Response(200, json={"title": "Imagine", "duration_s": 183})

transport = httpx.MockTransport(example_response)
with httpx.Client(transport=transport, timeout=10.0) as client:
    response = client.get("https://example.org/now-playing")
    response.raise_for_status()
    print(response.json())
```

demo11
httpx_client

SpiralTrain developer courses

requests and httpx

- `requests` provides a synchronous client and requires an explicit timeout
- `httpx` also supports async clients, HTTP/2 and mock transports
- Both return response objects with status, headers and body data

demo12
requests_or_httpx

SpiralTrain developer courses

typer

- `typer` derives arguments, conversions and help from function signatures
- `Annotated` supplies option descriptions
- `@app.command()` registers a function as a subcommand

```
@app.command("list")
def list_tracks(
    genre: Annotated[str | None, typer.Option("--genre")] = None,
    limit: Annotated[int, typer.Option("--limit")] = 5,
) -> None:
    """List tracks, optionally filtered by genre."""
    show_tracks(genre, limit)
```

demo13
typer_cli

SpiralTrain developer courses

typer Trade-offs

- `argparse` is in the standard library; `typer` is a dependency
- Both support positional arguments, options and subcommands
- `typer.Exit(code=...)` sets an exit status
- The playlist `list`, `stats` and `top` commands have the same jobs in both APIs

SpiralTrain developer courses

Finding Libraries

- PyPI lists Python distributions and their project links
- A dependency declaration records the requested package
- The lockfile records the resolved versions

```
uv add rich
```

SpiralTrain developer courses

Judging a Library

- Read its documentation, changelog and licence
- Check maintenance activity and dependency requirements
- Installed distribution metadata provides version and project information

```
from importlib import metadata

print(metadata.distribution("rich").version)
```

demo14
judging_a_library

SpiralTrain developer courses

Summary : Modern Toolbelt

- Wrap loops with progress displays
- Use `rich` for structured reports and diagnostic detail
- Test HTTP examples with an offline transport
- Build a CLI with annotated functions when using `typer`

- Inheritance
- Command-line Programs
- Modern Toolbelt
- ▶ **Web Scraping**
- SQLAlchemy
- NumPy
- Pandas
- GUI Toolkits

APPENDIX

- Running Other Programs

- Web Scraping
- Fetching a Page
- Status Codes and Timeouts
- BeautifulSoup
- find and find_all
- Text and Attributes
- CSS Selectors
- Navigating the Tree
- Missing Elements
- HTML Tables
- Extracting Records
- SQLite Schema
- Parameterised Inserts
- SQL Injection
- Upsert
- Scrape Pipeline
- robots.txt and Rate Limiting
- Scraping versus an API
- Database Handover

Web Scraping

- **Every scraper is the same three steps, in the same order :**
  - Fetch, parse, store
- **Each stage has an independent responsibility :**
  - Each step can be tested and re-run without the others
  - Only the middle step knows anything about HTML



Fetching a Page

- **`requests.get` returns a response object, not a string :**
  - `.text` is the decoded body, `.content` is the raw bytes
- **Two arguments matter far more than the rest :**
  - `timeout`, because there is no default and a hung server hangs you
  - `headers`, to say which program is asking

```
response = requests.get(URL, timeout=10, headers={"User-Agent": "SpiralTrain-Day4/1.0"})
```

demo01
fetching_pages

SpiralTrain developer courses

Status Codes and Timeouts

- **The status code is the first thing to check :**
  - `200` fine, `301` and `302` redirects that `requests` follows for you
  - `404` gone, `403` refused, `429` slow down, `5xx` their problem
- **`raise_for_status` turns any `4xx` or `5xx` into an exception :**
  - Nothing happens for `2xx` and `3xx`, so it is safe to always call
- **`response.encoding` can be set when a page is decoded wrongly**

```
200 OK                    passes, response.ok is True
404 Not Found             raises HTTPError
429 Too Many Requests     raises HTTPError
```

SpiralTrain developer courses

BeautifulSoup

- **`BeautifulSoup` takes the markup and the name of a parser :**
  - `"lxml"` is fast and forgiving, and is installed for this course
  - `"html.parser"` is built into Python and needs nothing extra
- **The result is a tree you can search, not a string :**
  - Real pages are malformed, and the parser quietly repairs them
- **`response.text` is already decoded, so it goes straight in :**
  - The server declares the encoding, and a price of £51.77 proves it worked

```
soup = BeautifulSoup(html, "lxml")

# Page titles are full of markup whitespace, so collapse it rather than
# trusting what the page hands you.
print(" ".join(soup.title.text.split()))
```

demo02
making_soup

SpiralTrain developer courses

find and find_all

- `find` **returns the first match, or** `None` **:**
  - `find_all` returns a list, which may be empty
- **Both take a tag name, and keyword arguments for attributes :**
  - `class_` has the underscore because `class` is a Python keyword
- **Searching a tag searches inside that tag only**

```
for article in soup.find_all("article", class_="product_pod"):
    title = article.find("h3").find("a")["title"]
    if title == "Sharp Objects":
        price = article.find("p", class_="price_color").text
        # The page spells the rating as a class name, not a number: markup is
        # written for a browser, not for you.
        stars = article.find("p", class_="star-rating")["class"][1]
        print(f"{RATINGS[stars]}  {price:>8}  {title}")
```

demo03
finding_the_books

SpiralTrain developer courses

Text and Attributes

- `.text` collects every string inside a tag :
  - `get_text(strip=True)` is the same thing with the whitespace tidied
- Attributes are read like a dictionary :
  - `tag["href"]` raises `KeyError` when the attribute is absent
  - `tag.get("href")` returns `None`, or a default you supply
- `class` comes back as a list, because an element can carry several

```
stars = pod.find("p", class_="star-rating")
print(stars["class"])              # ['star-rating', 'Four']
print(stars.get("data-value"))     # None, the attribute is not there
```

SpiralTrain developer courses

CSS Selectors

- **`select` and `select_one` take one selector string :**
  - The same language a stylesheet or the browser inspector uses
- **A whole path through the document fits in one argument :**
  - `article.product_pod h3 a` means a inside h3 inside that article
  - `p.star-rating.Four` means both classes on one element

```
print(len(soup.select("article.product_pod h3 a")), "title links")
print(soup.select("article.product_pod h3 a")[1]["title"])

print(soup.select_one("ul.breadcrumb li.active").text.strip())

# Two class names on one element: both, with no space between them.
print(len(soup.select("p.star-rating.Four")), "books rated four")

print(len(soup.select("div.image_container > a > img[alt]")), "thumbnails")
```

demo04
css_selectors

SpiralTrain developer courses

Navigating the Tree

- **Start at something easy to find and move from there :**
  - `.parent` is one step up, `.find_parent("article")` goes up until it matches
  - `.find_next_sibling("p")` skips the whitespace between tags
- **Useful when the thing you want has no class of its own :**
  - A price is easy to find, the module it belongs to is upward from it

```
price = soup.select_one("p.price_color")
print(price.parent.name)

pod = price.find_parent("article")
print(pod.select_one("h3 a")["title"])

print(price.find_next_sibling("p").get_text(strip=True))
```

demo05
walking_the_tree

SpiralTrain developer courses

Missing Elements

- **Real pages are inconsistent, and a scraper meets all of it :**
  - The attribute missing on one row in five hundred
  - The module that has no price because the item is sold out
- **Decide per field what absence means :**
  - `.get("href")` for optional, square brackets for required
  - A required field that is missing should fail loudly, not silently

```
url = link["href"]                          # required, so let it raise
cover = image.get("data-src", "")           # optional, so a default is fine
if pod.find("p", class_="price_color") is None:
    continue                                # no price, no record
```

SpiralTrain developer courses

HTML Tables

- **A table is the easiest markup to scrape, and the most common :**
  - One header row gives the keys, one body row gives the values
- **`zip(headers, cells, strict=True)` refuses to guess :**
  - A short row raises instead of silently shifting every column

```
headers = [cell.get_text(strip=True) for cell in table.select("tr th")]

rows = []
for row in table.select("tbody tr"):
    cells = [cell.get_text(strip=True) for cell in row.find_all("td")]
    rows.append(dict(zip(headers, cells, strict=True)))
```

demo06
parsing_a_table

SpiralTrain developer courses

Extracting Records

- **One function, one page, a list of dictionaries out :**
  - After this line the markup has stopped mattering
- **Convert while parsing, not later :**
  - A price becomes a `float`, a rating becomes an `int`
  - A relative link becomes an absolute one, with `urljoin`

```
soup = BeautifulSoup(html, "lxml")

books = []
for pod in soup.select("article.product_pod"):
    link = pod.select_one("h3 a")
    books.append({
        "title": link["title"],
        "price_gbp": float(re.sub(r"[^\d.]", "", pod.select_one("p.price_color").text)),
        "url": urljoin(URL, link["href"]),
    })
```

demo07
extracting_records

SQLite Schema

- **The same care as any other schema :**
  - One column per field, with a type, and `NOT NULL` where it is true
  - A natural key, `UNIQUE`, so the same page cannot arrive twice
- **`first_seen` and `last_seen` make a re-run informative**

```
CREATE TABLE IF NOT EXISTS books (
    id           INTEGER PRIMARY KEY,
    title        TEXT    NOT NULL,
    price_gbp    REAL    NOT NULL,
    rating       INTEGER NOT NULL,
    url          TEXT    NOT NULL UNIQUE,
    last_seen    TEXT    NOT NULL
);
```

SpiralTrain developer courses

Parameterised Inserts

- **Placeholders keep the values out of the SQL text :**
  - `?` by position, or `:name` by name
- **Named placeholders take a dictionary as it is :**
  - Which is exactly what the parse step produced
- **`executemany` sends the whole list in one call**

```
conn = sqlite3.connect(DB)
conn.executescript(SCHEMA)
# Named placeholders take the dictionaries exactly as they already are.
conn.executemany(INSERT, books)
conn.commit()
```

SQL Injection

- **The database is not the only thing that reads the string :**
  - A title containing an apostrophe ends the quoted value early
  - The following text is parsed as SQL
- **One book on the page proves it :**

```
title = "The Bachelor Girl's Guide to Murder"

f"SELECT ... WHERE title = '{title}'"   sqlite3.OperationalError: near "s"
"SELECT ... WHERE title = ?", (title,)  found 1 row
```

- **Parameters keep values separate from the SQL statement**

SpiralTrain developer courses

Upsert

- **A scrape that only works the first time is not finished :**
  - The job will be re-run, after a crash, or every night
- **`INSERT ... ON CONFLICT DO UPDATE` is one statement for both cases :**
  - `excluded` is the row that was about to be inserted
  - Columns left out of the `SET` keep the value they had

```
INSERT INTO books (title, price_gbp, url, first_seen, last_seen)
VALUES (:title, :price_gbp, :url, :seen_at, :seen_at)
ON CONFLICT(url) DO UPDATE SET
    price_gbp = excluded.price_gbp,
    last_seen = excluded.last_seen
```

demo09
upsert

SpiralTrain developer courses

Scrape Pipeline

- **One script, read top to bottom, in four labelled sections :**
  - Fetch the page, parse the records, store them, report on the table
- **Nothing is wrapped in a function it does not need :**
  - The whole file is the pipeline, and the order on screen is the order it runs
- **Running it a second time changes `last_seen` and nothing else**

```
uv run demos/04-web-scraping/10_scrape_pipeline.py
```

demo10
scrape_pipeline

SpiralTrain developer courses

robots.txt and Rate Limiting

- `robots.txt` describes which paths a crawler may request.
- `RobotFileParser` reads the rules; this example uses saved text.

```
from urllib.robotparser import RobotFileParser

ROBOTS = "User-agent: *\nDisallow: /basket/\nCrawl-delay: 5\n"
AGENT = "TrainingDemo/1.0"
url = "https://books.toscrape.com/basket/"
rules = RobotFileParser()
rules.parse(ROBOTS.splitlines())
print(rules.can_fetch(AGENT, url))  # False
print(rules.crawl_delay(AGENT))  # 5
```

- Identify the scraper with a `User-Agent` and contact details.
- Respect the site's rules, pause between requests and reuse cached pages.

SpiralTrain developer courses

Scraping versus an API

- **Scraping is the last resort, not the first :**
  - An API returns JSON that does not change when the design does
  - A CSV or data export is better still, and often already there
- **Signs it is time to stop :**
  - The page builds itself with JavaScript, so the markup is empty
  - There is a login, a CAPTCHA, or a rate limit aimed at you
  - The markup changes weekly and the parser breaks weekly
- **Look for `/api/`, an RSS feed, or a documented download first**

SpiralTrain developer courses

Database Handover

- **The database is the handover point :**
  - Pandas reads it back with `read_sql`
  - A GUI can put a window on the same table
- **A checked in `.sql` file rebuilds it without the scraper :**
  - So the later blocks work even if nothing was ever scraped

```
conn = sqlite3.connect(DB)
conn.executescript((DATA / "books-seed.sql").read_text(encoding="utf-8"))
conn.commit()
```

demo12
rebuilding_from_seed

SpiralTrain developer courses

Summary : Web Scraping

- Fetch, parse and store live in separate functions
- Every request has a timeout and a `User-Agent`
- Parsing is a separate step, so it can be re-run without re-fetching
- Values are converted at the parse step, not at the database
- Inserts are parameterised, and the re-run is an upsert
- The pace respects `robots.txt`, and the log says who you are

- Inheritance
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- ▶ **SQLAlchemy**
- NumPy
- Pandas
- GUI Toolkits

APPENDIX

- Running Other Programs

- SQLAlchemy
- sqlite3 and SQLAlchemy
- Engine and URL
- begin and connect
- Bound Parameters
- Table Definitions
- Core Queries
- Declarative Models
- Session
- ORM Queries
- Relationships
- Joins and Relationships
- Reflection
- SQLAlchemy and Pandas
- text, Core and ORM

SQLAlchemy

- **The scraping module talked to SQLite with `sqlite3` :**
  - Which is the driver, and drivers are database-specific
- **SQLAlchemy sits above the driver :**
  - One way of working, whichever database is underneath
- **It has two halves, and both are useful :**
  - `Core`, which builds SQL out of Python objects
  - The `ORM`, which maps rows onto classes
- **Nothing here replaces knowing SQL :**
  - It repays knowing SQL, which is the point

SpiralTrain developer courses

sqlite3 and SQLAlchemy

- **The same call, against a different database :**
  - Change the URL, not the program
- **Parameters have names instead of positions**
- **Rows can come back as objects rather than tuples**
- **The query is a value you can build up, print and reuse**

```
found = connection.execute(
    text("SELECT title, price FROM book WHERE price > :floor"), {"floor": 25}
)
print("sqlalchemy :", found.all())
```

SpiralTrain developer courses

Engine and URL

- **An engine is a factory and a connection pool :**
  - Make one per database, once, and keep it
  - It does not open a connection until something asks
- **The URL says everything about where the data lives :**

```
dialect+driver://user:password@host/database

sqlite:///:memory:
sqlite:///data/library.db
postgresql+psycopg://feiko@localhost/inventory
```

- **SQLite needs no server, so the middle of the URL is empty**

demo02
engine_and_connection

SpiralTrain developer courses

begin and connect

- `engine.begin()` **opens a transaction :**
  - Commits when the module ends, rolls back if it raises
- `engine.connect()` **gives you a connection and nothing else :**
  - Anything written is discarded unless you commit it yourself
- **Prefer** `begin()` **for anything that writes**

```
with engine.begin() as connection:
    connection.execute(text("INSERT INTO note VALUES ('kept')"))

with engine.connect() as connection:
    rows = connection.execute(text("SELECT body FROM note")).scalars().all()
```

SpiralTrain developer courses

Bound Parameters

- **The parameter is sent separately from the statement :**
  - The database never parses the value as SQL
- **Named, so there are no question marks to count**
- **The scraping module uses the same separation of SQL and values**

```
statement = text("SELECT title FROM book WHERE rating >= :floor")
connection.execute(statement, {"floor": 3})
```

- **A formatted query breaks on ordinary data long before it is attacked :**

```
SELECT rating FROM book WHERE title = 'The Widow's Tale'
```

demo03
parameters

SpiralTrain developer courses

Table Definitions

- **Core describes a table as a Python object :**
  - Columns, types, constraints, all as values
- **`MetaData` is the catalogue every table registers in**
- **`create_all()` issues `CREATE TABLE` for whatever is missing**

```
book = Table(
    "book",
    metadata,
    Column("id", Integer, primary_key=True),
    Column("title", String(200), nullable=False),
    Column("price_gbp", Float, nullable=False),
)
```

demo04
describing_a_table

SpiralTrain developer courses

Core Queries

- `select()` returns something you can hold, print and add to :
  - `.where()`, `.order_by()`, `.limit()` each return a new query
- Column access is `table.c.name`
- Printing the query shows the SQL it will send

```
statement = select(book.c.title, book.c.price_gbp).where(book.c.rating >= 4)
print(statement)

with engine.connect() as connection:
    for title, price in connection.execute(statement):
        print(f"{title:<24} {price:>6.2f}")
```

demo05
core_insert_and_select

SpiralTrain developer courses

Declarative Models

- **The ORM maps a class onto a table :**
  - One class, one table, one instance per row
- **Every model inherits from a `DeclarativeBase` :**
  - Which is block one doing real work
- **`Mapped[int]` is an ordinary annotation, so editors understand it**

```
class Book(Base):
    __tablename__ = "book"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    price_gbp: Mapped[float] = mapped_column(Float)
```

demo06
declarative_model

Session

- **A `Session` is a unit of work, not a connection :**
  - Add objects to it, then commit once
- **Generated ids become available when inserts are flushed**
- **The session tracks what it loaded :**
  - Change an attribute and there is no `UPDATE` to write
- **`flush()` sends pending writes inside the current transaction**
- **`commit()` makes the transaction permanent**

```
with Session(engine) as session:
    session.add_all([first, second])
    session.flush()
    session.commit()

    first.price_gbp = 41.99
    session.commit()
```

demo07
session_and_persistence

SpiralTrain developer courses

ORM Queries

- The same `select()`, but the rows are objects
- `scalars()` unwraps a one-column result into the thing itself
- Comparisons build SQL rather than returning a boolean :
  - `Book.rating >= 4` is a fragment of a `WHERE` clause

```
for found in session.scalars(select(Book).where(Book.rating >= 4)):
    print(found)
```

```
for rating, count, mean in session.execute(
    select(Book.rating, func.count(), func.avg(Book.price_gbp)).group_by(Book.rating)
):
    print(f"{rating}* {count:>3} {mean:>8.2f}")
```

demo08
orm_queries

SpiralTrain developer courses

Relationships

- **A foreign key becomes an attribute on both sides :**
  - `category.books` is a list, `book.category` is an object
- **`back_populates` names the attribute pointing the other way**
- **Appending to the list is what creates the link :**
  - No ids written by hand, and the insert order is worked out

```
class Category(Base):
    books: Mapped[list[Book]] = relationship(back_populates="category")

mystery.books.append(Book(title="Sharp Objects"))
session.add(mystery)
```

Joins and Relationships

- **As a join :**
  - One query
  - Explicit, and familiar
  - What the database does anyway

```
joined = select(Book.title, Category.name).join(Book.category)
print(joined)
for title, category in session.execute(joined):
    print(f"{title:<18} {category}")

for found in session.scalars(select(Book)):
    print(f"{found.title:<18} {found.category.name}")
```

- **As an attribute :**
  - **Reads better**
  - **Issues a query per object**
  - **Easy to do by accident**

demo10
joins

SpiralTrain developer courses

Reflection

- **Reflection reads the schema out of an existing database :**
  - Rather than describing it in Python first
- **Which is most databases, most of the time**

```
metadata = MetaData()
metadata.reflect(bind=engine)

books = metadata.tables["books"]
print([column.name for column in books.columns])
```

- **The demo points at the database the scraping module filled in**

demo11
reading_the_scraped_db

SpiralTrain developer courses

SQLAlchemy and Pandas

- **`read_sql` takes an engine anywhere it takes a connection :**
  - So the portable half comes along for free
- **The grouping can happen in either place :**
  - In the database, or in the DataFrame afterwards
- **For twenty rows it makes no difference :**
  - For twenty million, let the database do it

```
engine = create_engine(f"sqlite:///{DB}")
frame = pd.read_sql("SELECT title, price_gbp, rating FROM books", engine)
```

demo12
with_pandas

SpiralTrain developer courses

text, Core and ORM

- **`text()` when the SQL is the clearest way to say it :**
  - Or when it is already written and correct
- **`Core` when the query is assembled from parts, or must be portable**
- **The `ORM` when rows are things the program keeps hold of and changes**
- **These are not levels, and not a progression :**
  - Plenty of good programs use all three in one file

demo13
core_or_orm

SpiralTrain developer courses

Summary : SQLAlchemy

- SQLAlchemy sits above the driver, not above SQL
- An engine is made once and carries the URL :
  - Use `begin()` for anything that writes
- Core builds queries as objects you can print and compose
- The ORM maps a class onto a table, with a `Session` as the unit of work
- Relationships turn a foreign key into an attribute on both sides
- Reflection reads a schema somebody else designed
- An engine drops straight into `pandas.read_sql`

- Inheritance
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- **NumPy**
- Pandas
- GUI Toolkits

APPENDIX

- Running Other Programs

- NumPy Arrays
- Memory Use
- Speed
- Creating Arrays
- Array Data Types
- Shape and Dimensions
- Reshaping and Transposing
- Indexing and Slicing
- Views and Copies
- Vectorised Arithmetic
- Boolean Masking
- np.where
- Broadcasting
- Aggregations
- axis Argument
- Random Numbers
- Reading a CSV

SpiralTrain developer courses

NumPy Arrays

- An array holds values of one dtype in a regular shape
- Arithmetic applies to every element without a Python loop
- `np.sin` can compute samples of an audio tone

```
times = np.arange(8) / 8000
samples = np.sin(2 * np.pi * 440 * times)
print(samples.round(3))
print(samples * 0.5)
```

demo01
first_array

SpiralTrain developer courses

Memory Use

- A Python list holds references to Python objects
- A numeric array stores the numbers in a contiguous buffer
- `nbytes` measures the array buffer; `getsizeof` measures Python objects

```
count = 60 * 44100
samples = [math.sin(2 * math.pi * 440 * i / 44100) for i in range(count)]
tone = np.array(samples)
list_bytes = sys.getsizeof(samples) + sum(sys.getsizeof(x) for x in samples)
print(f"list: {list_bytes:,} bytes")
print(f"array: {tone.nbytes:,} bytes")
```

demo02
memory_footprint

Speed

- NumPy evaluates array arithmetic in compiled code
- A Python loop evaluates one sample at a time
- Compare equivalent results before comparing elapsed time

```
count = 60 * 44100
start = time.perf_counter()
samples = [math.sin(2 * math.pi * 440 * i / 44100) for i in range(count)]
print(f"Python: {time.perf_counter() - start:.3f} s")
start = time.perf_counter()
times = np.arange(count) / 44100
tone = np.sin(2 * np.pi * 440 * times)
print(f"NumPy: {time.perf_counter() - start:.3f} s")
print(np.allclose(samples, tone))
```

demo03
speed_comparison

SpiralTrain developer courses

Creating Arrays

- `array` converts an existing sequence
- `arange` selects a step; `linspace` selects a sample count
- `zeros`, `ones` and `full` fill a given shape

```
print(np.array([183, 126, 146]))
print(np.arange(0, 8, 2))
print(np.linspace(0.0, 1.0, 5))
print(np.zeros((2, 4)))
print(np.ones(3, dtype=int))
print(np.full(3, 0.5))
```

demo04
making_arrays

SpiralTrain developer courses

Array Data Types

- `dtype` determines the representation of each element
- Request a dtype at construction or convert with `astype`
- Integer conversions discard fractional parts; narrow integers have limits

```
samples = np.array([0.1, 0.5, -0.4], dtype=np.float32)
print(samples.dtype)
print(samples.astype(np.int16))
print(np.iinfo(np.int16).min, np.iinfo(np.int16).max)
counts = np.array([183, 126, 146], dtype=np.int64)
print(counts.dtype, counts.itemsize)
```

demo05
dtypes

SpiralTrain developer courses

Shape and Dimensions

- An RGB image has height, width and colour-channel dimensions
- `.shape` gives each dimension's length; `.ndim` counts dimensions
- `.size` counts all elements

```
print(photo.shape)
print(photo.ndim)
print(photo.size)
print(photo[0, 0].shape)
```

demo06
shape_and_ndim

SpiralTrain developer courses

Reshaping and Transposing

- `reshape` changes dimensions while preserving the element count
- `-1` asks NumPy to calculate one dimension
- `transpose` rearranges axes; `.T` reverses them

```
pixels = photo.reshape(-1, 3)
print(pixels.shape)
print(np.shares_memory(pixels, photo))
print(photo.transpose(1, 0, 2).shape)
print(pixels.T.shape)
```

demo07
reshape_and_transpose

SpiralTrain developer courses

Indexing and Slicing

- Give one index or slice per dimension, separated by commas
- A slice excludes its stop position
- A negative step reverses an axis

```
print(photo[24, 32])
crop = photo[12:36, 16:48]
print(crop.shape)
flipped = photo[::-1]
print(np.array_equal(flipped[0], photo[-1]))
print(photo[:, :, 0].shape)
```

demo08
indexing_and_slicing

SpiralTrain developer courses

Views and Copies

- Basic slicing returns a view of the same buffer
- Writing through a view also changes the original
- `.copy()` makes an independent buffer; advanced indexing also copies

```
crop = photo[12:36, 16:48]
detached = crop.copy()
crop[0, 0] = [1.0, 0.0, 0.0]
print(photo[12, 16])
print(detached[0, 0])
print(np.shares_memory(crop, photo))
print(np.shares_memory(detached, photo))
print(np.shares_memory(photo[[0, 1]], photo))
```

demo09
views_and_copies

SpiralTrain developer courses

Vectorised Arithmetic

- A scalar applies to every element
- Matching shapes combine element by element
- Functions such as `sqrt`, `sin` and `log10` also accept arrays

```
times = np.arange(8) / 8000
tone = np.sin(2 * np.pi * 440 * times)
quiet = tone * 0.5
print(quiet.round(3))
print(np.sqrt(tone ** 2).round(3))
print((tone - quiet).round(3))
```

demo10
vectorised_arithmetic

SpiralTrain developer courses

Boolean Masking

- A comparison creates an array of booleans
- Indexing with a mask keeps matching values
- `&` and `|` combine conditions; each comparison needs parentheses

```
tone = np.array([0.0, 0.2, 0.9, -0.8, 0.0, -0.1])
silent = np.abs(tone) < 0.05
print(tone[silent])
print(silent.sum(), silent.any(), silent.all())
print(np.nonzero(silent)[0])
print(tone[(tone > -0.5) & (tone < 0.5)])
```

demo11
boolean_masking

np.where

- `where(condition, yes, no)` chooses a value for each element
- Each branch can be an array or a scalar
- Nesting a second `where` supports three outcomes

```
tone = np.array([-0.9, -0.2, 0.0, 0.4, 0.9])
clipped = np.where(tone > 0.5, 0.5, np.where(tone < -0.5, -0.5, tone))
print(clipped)
print(np.where(np.abs(tone) < 0.05, "silence", "sound"))
print(np.where(tone > 0.5)[0])
```

demo12
np_where

SpiralTrain developer courses

Broadcasting

- Shapes align from the right
- Dimensions must be equal or one of them must be `1`
- Three channel factors apply to every pixel in an RGB photo

```
factors = np.array([1.1, 1.0, 0.9])
corrected = np.clip(photo * factors, 0.0, 1.0)
print(photo.shape, factors.shape, corrected.shape)
print(np.broadcast_shapes((48, 64, 3), (3,)))
print(np.broadcast_shapes((3, 1), (4,)))
```

demo13
broadcasting

Aggregations

- Reductions produce totals, averages or extreme values
- `argmax` gives a position; `unravel_index` converts it to coordinates
- Median and percentiles describe the distribution

```
print(plays.sum(), plays.mean().round(2))
print(plays.min(), plays.max(), plays.std().round(2))
print(np.median(plays), np.percentile(plays, 95))
day, hour = np.unravel_index(np.argmax(plays), plays.shape)
print(f"peak: day {day}, hour {hour:02d}")
```

demo14
aggregations

SpiralTrain developer courses

axis Argument

- `axis` names the dimension removed by the reduction
- A days-by-hours grid reduces to hours with `axis=0`
- `axis=1` removes the hours and leaves a value per day
- `keepdims=True` retains the reduced dimension with length one

```
plays = np.array([[1, 2, 3, 4], [2, 0, 1, 3], [0, 1, 2, 1]])
print(plays.sum())
print(plays.sum(axis=0))
print(plays.sum(axis=1))
print(plays.sum(axis=1, keepdims=True).shape)
```

demo15
the_axis_argument

SpiralTrain developer courses

Random Numbers

- `default_rng(seed)` produces reproducible pseudorandom draws
- `choice` samples existing values; `permutation` returns a shuffled copy
- `shuffle` changes its input in place

```
rng = np.random.default_rng(7)
titles = np.array(["Imagine", "Yesterday", "Respect", "Toxic"])
print(rng.permutation(titles))
print(rng.choice(titles, size=2, replace=False))
print(rng.integers(0, 10, size=4))
print(rng.random(3).round(2))
print(rng.normal(0.0, 0.1, size=3).round(2))
rng.shuffle(titles)
print(titles)
```

demo16
random_numbers

SpiralTrain developer courses

Reading a CSV

- `loadtxt` reads regular numeric data
- `skiprows` excludes the header; `usecols` excludes the day labels
- `savetxt` writes an array back to a file

```
print(plays.shape)
print(plays.sum(axis=0))
print(plays.sum(axis=1))
print(int(np.argmax(plays.sum(axis=0))))
np.savetxt(DATA / "plays-hourly-mean.csv", plays.mean(axis=0),
           delimiter=",", fmt="%.2f")
```

demo17
csv_into_array

SpiralTrain developer courses

Summary : NumPy

- Arrays have a dtype and a regular shape
- Slices usually share memory; copies own independent buffers
- Arithmetic, masks and `where` operate on whole arrays
- Broadcasting aligns compatible dimensions from the right
- Reductions remove the selected axes; seeded generators repeat their draws

- Inheritance
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- NumPy
- **Pandas**
- GUI Toolkits

APPENDIX

- Running Other Programs

- Pandas
- Series
- DataFrame
- The Index
- Reading a CSV
- Reading a Spreadsheet
- head and tail
- info and describe
- Selecting Columns
- loc
- iloc
- loc and iloc
- Filtering Rows
- Combining Filters
- Adding a Column
- Deriving a Column
- Sorting
- groupby
- groupby Aggregation
- Named Aggregation
- merge
- Join Types
- Missing Values
- read_sql
- to_sql
- Writing Files
- Plotting

Pandas

- `pandas` works with labelled tables :
  - `Series` represents a column; `DataFrame` represents a table
- CSV files, spreadsheets and query results share the same operations
- Column operations use NumPy arrays and other array implementations
- A typical report reads, inspects, filters, summarises and exports data

```
import pandas as pd
```

Series

- A `Series` has one index label per value
- Arithmetic and comparisons apply to every value
- `.index` gives the labels; `.to_numpy()` gives the values

```
durations = pd.Series([183, 126, 146],
                      index=["Imagine", "Yesterday", "Respect"])
print(durations["Imagine"])
print(durations / 60)
print(durations > 180)
print(durations.index.tolist())
print(durations.to_numpy())
```

DataFrame

- A `DataFrame` has named columns with independent types
- A dictionary supplies one list per column
- `.shape` gives rows and columns; `.columns` gives their names

```
tracks = pd.DataFrame({
    "title": ["Imagine", "Yesterday", "Respect"],
    "artist": ["John Lennon", "The Beatles", "Aretha Franklin"],
    "duration_s": [183, 126, 146],
})
print(tracks)
print(tracks.shape)
print(tracks["duration_s"])
```

demo02
dataframe_basics

The Index

- `set_index` promotes a column to row labels
- Filtering and sorting retain the labels
- Arithmetic aligns matching labels even when their positions differ

```
by_title = tracks.set_index("title")
print(by_title.loc["Imagine", ["artist", "duration_s"]])
durations = pd.Series([183, 126], index=["Imagine", "Yesterday"])
limits = pd.Series([180, 240], index=["Yesterday", "Imagine"])
print(limits - durations)
```

demo03
the_index

SpiralTrain developer courses

Reading a CSV

- `read_csv` infers column types from the file
- Useful options select a delimiter, columns and row labels :
  - `sep=`, `usecols=`, `index_col=`
- `parse_dates=` converts named date columns; `na_values=` adds missing-value markers

```
tracks = pd.read_csv(DATA / "tracks.csv", index_col="title",
                     parse_dates=["last_played"])
print(tracks[["artist", "duration_s", "last_played"]].head(3))
```

demo04
read_csv

SpiralTrain developer courses

Reading a Spreadsheet

- `read_excel` reads a sheet into a `DataFrame`
- `sheet_name=` selects the sheet by name or position
- `ExcelFile(...).sheet_names` lists the available sheets

```
tracks = pd.read_excel(DATA / "tracks.xlsx", sheet_name="tracks")
print(pd.ExcelFile(DATA / "tracks.xlsx").sheet_names)
print(tracks[["title", "genre", "plays"]].head(3))
```

demo05
read_excel

SpiralTrain developer courses

head and tail

- `head(n)` and `tail(n)` select rows at either end
- `sample(n, random_state=...)` gives a reproducible sample
- A sample can reveal values that the first rows do not contain

```
print(tracks[["genre", "plays"]].head(2))
print(tracks[["genre", "plays"]].tail(2))
print(tracks[["genre", "plays"]].sample(3, random_state=0))
```

demo06
head_and_tail

SpiralTrain developer courses

info and describe

- `info()` reports names, dtypes and non-missing counts
- `describe()` summarises numeric columns :
  - Count, mean, standard deviation, quartiles and range
- Date parsing changes a text column into a datetime dtype

```
tracks.info()
print(tracks[["duration_s", "plays"]].describe().round(1))
```

demo07
info_and_describe

SpiralTrain developer courses

Selecting Columns

- A column name selects a `Series`
- A list of column names selects a `DataFrame`
- The list also determines the report column order

```
print(tracks["plays"].head(3))
print(tracks[["plays"]].head(3))
print(tracks[["genre", "artist", "duration_s"]].head(3))
```

demo08
selecting_columns

SpiralTrain developer courses

loc

- **`loc[rows, columns]` selects by labels :**
  - One label, a list of labels, or a label slice
- **Label slices include both ends**

```
print(tracks.loc["Imagine", "duration_s"])
print(tracks.loc["Imagine", ["artist", "genre"]])
print(tracks.loc["Imagine":"Respect", "year":"plays"])
print(tracks.loc[:, ["genre", "plays"]].head(3))
```

demo09
loc_by_label

SpiralTrain developer courses

iloc

- `iloc[rows, columns]` selects by integer positions
- Integer slices exclude the stop position
- Moving rows changes what an integer position selects

```
print(tracks.iloc[0, 3])
print(tracks.iloc[:3, :2])
print(tracks.iloc[-2:, :2])
print(tracks.iloc[[0, 5, 2], [0, 1]])
```

demo10
iloc_by_position

loc and iloc

- `loc` uses labels and includes the slice endpoint
- `iloc` uses zero-based positions and excludes the endpoint
- Both put rows before columns
- Plain `frame[...]` selects columns; a boolean mask selects rows

```
print(tracks.loc["Imagine", "duration_s"])
print(tracks.iloc[0, 3])
```

SpiralTrain developer courses

Filtering Rows

- A column comparison produces a boolean `Series`
- Indexing with that mask keeps rows where it is `True`
- Summing a mask counts matching rows

```
popular = tracks["plays"] >= 50
print(tracks[popular][["genre", "plays"]].head(4))
print(int(popular.sum()))
print(tracks[tracks["genre"] == "Soul"][["plays"]])
```

demo11
boolean_filter

SpiralTrain developer courses

Combining Filters

- `&`, `|` and `~` combine boolean masks
- Each comparison needs parentheses
- `isin` checks membership; `between` checks an inclusive interval

```
mask = (tracks["plays"] >= 50) & (tracks["duration_s"] < 240)
print(tracks[mask][["duration_s", "plays"]].head(4))
print(tracks[tracks["genre"].isin(["Soul", "Country"])][["genre"]].head(4))
print(tracks[tracks["year"].between(1980, 1999)][["year"]].head(4))
print(tracks[~tracks["genre"].isin(["Rock", "Pop"])][["genre"]].head(4))
```

demo12
combining_filters

SpiralTrain developer courses

Adding a Column

- Assigning a new column name creates the column
- A scalar applies to every row; a computed `Series` aligns by index
- `assign` returns a new frame; `drop(columns=...)` removes columns

```
tracks["available"] = True
tracks["listened_s"] = tracks["duration_s"] * tracks["plays"]
report = tracks.assign(minutes=tracks["listened_s"] / 60)
print(report[["plays", "minutes"]].head(3).round(1))
print(tracks.drop(columns=["available"]).columns.tolist())
```

demo13
adding_columns

SpiralTrain developer courses

Deriving a Column

- `.str` applies string operations down a column
- `.map(dictionary)` translates values to labels
- `np.where` chooses between values for each row

```
tracks["initial"] = tracks["artist"].str.upper().str[:1]
tracks["style"] = tracks["genre"].map({"Rock": "guitar", "Pop": "vocal"})
tracks["status"] = np.where(tracks["plays"] > 50, "favourite", "occasional")
tracks.loc[tracks["last_played"].isna(), "status"] = "never"
print(tracks[["initial", "style", "status"]].head(6))
```

demo14
deriving_columns

SpiralTrain developer courses

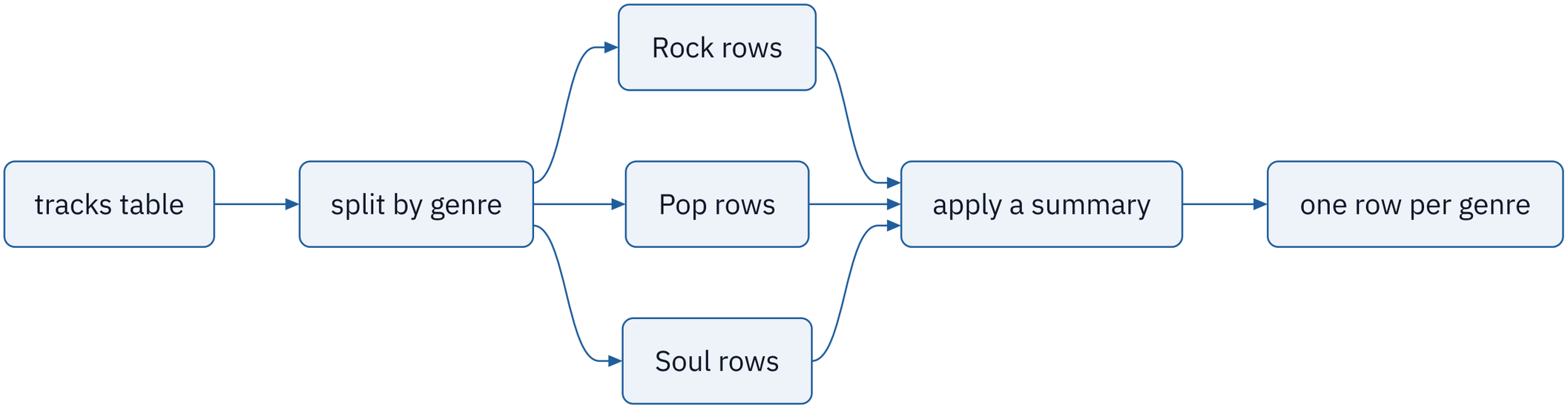
Sorting

- `sort_values` takes one column or a list of columns
- `ascending` accepts one boolean or a boolean per column
- `nlargest` and `nsmallest` select extreme rows; `sort_index` orders labels

```
print(tracks.sort_values("plays", ascending=False)[["plays"]].head(3))
print(tracks.sort_values(["genre", "plays"], ascending=[True, False])
      [["genre", "plays"]].head(4))
print(tracks.nlargest(3, "plays")[["plays"]])
print(tracks.nsmallest(2, "duration_s")[["duration_s"]])
print(tracks.sort_index()[["plays"]].head(2))
```

demo15
sort_values

groupby



- **Each distinct grouping value produces one summary row**

SpiralTrain developer courses

groupby Aggregation

- Select the grouping column, then the values and summary
- `agg` accepts a list of summaries or a dictionary per column
- The group label becomes the result index

```
print(tracks.groupby("genre")["plays"].sum())
print(tracks.groupby("genre")["duration_s"].mean().round(1))
print(tracks.groupby("genre").size())
print(tracks.groupby("genre")["plays"].agg(["count", "sum", "mean", "max"]))
print(tracks.groupby("genre").agg({"plays": "sum", "year": "min"}))
```

demo16
groupby_agg

SpiralTrain developer courses

Named Aggregation

- Each keyword names a result column
- Its value is `(source_column, summary)`
- `reset_index()` restores grouping labels as ordinary columns

```
per_genre = tracks.groupby("genre").agg(
    titles=("artist", "count"),
    total_plays=("plays", "sum"),
    mean_duration=("duration_s", "mean"),
).round(1).reset_index()
print(per_genre)
tracks["decade"] = tracks["year"] // 10 * 10
print(tracks.groupby(["genre", "decade"])["plays"].sum())
```

demo17
groupby_named_agg

SpiralTrain developer courses

merge

- `merge` combines frames on a shared key
- `on` names the shared column
- `left_on` and `right_on` support different key names

```
artists = pd.read_csv(DATA / "artists.csv")
joined = tracks.merge(artists, on="artist")
print(joined[["title", "artist", "country"]].head(3))
print(joined.groupby("country")["plays"].sum())
```

demo18
merge_on_key

SpiralTrain developer courses

Join Types

- `inner` keeps matching keys; `left` keeps every left row
- `right` keeps every right row; `outer` keeps both sides
- `indicator=True` records `both`, `left_only` or `right_only`

```
artists = pd.read_csv(DATA / "artists.csv")
for how in ("inner", "left", "right", "outer"):
    merged = tracks.merge(artists, on="artist", how=how)
    print(f"{how}: {len(merged)} rows")
outer = tracks.merge(artists, on="artist", how="outer", indicator=True)
print(outer.loc[outer["_merge"] != "both", ["title", "artist", "_merge"]])
```

demo19
merge_how

SpiralTrain developer courses

Missing Values

- `isna` recognises missing values; summing its mask counts them
- `dropna(subset=...)` removes rows missing selected columns
- `fillna` replaces missing values explicitly
- Summaries generally skip missing values

```
print(tracks.isna().sum())
print(tracks.loc[tracks["last_played"].isna(), ["artist", "last_played"]])
print(len(tracks.dropna(subset=["last_played"])))
filled = tracks.fillna({"last_played": "never"})
print(filled.loc["Hey Jude", "last_played"])
```

demo20
missing_data

read_sql

- `read_sql` returns a query result as a `DataFrame`
- A `sqlite3` connection can read the scraping module's books database
- Query parameters keep values separate from SQL

```
query = "SELECT title, price_gbp, rating FROM books WHERE price_gbp > ? ORDER BY price_gbp DESC"
print(pd.read_sql(query, conn, params=(45.0,)).to_string())
```

SpiralTrain developer courses

to_sql

- `to_sql` writes a frame to a database table
- `if_exists` selects `fail`, `replace` or `append`
- `index=False` excludes row labels from the stored table

```
conn = sqlite3.connect(DATA / "pandas-demo.db")
books = pd.read_csv(DATA / "books-sample.csv")
summary = books.groupby("rating", as_index=False)["price_gbp"].mean()
summary.to_sql("rating_summary", conn, if_exists="replace", index=False)
conn.commit()
print(pd.read_sql("SELECT * FROM rating_summary", conn))
conn.close()
```

demo22
to_sql

SpiralTrain developer courses

Writing Files

- `to_csv` and `to_excel` export frames
- `index=False` excludes row labels when they are not report data
- `ExcelWriter` writes several sheets into one workbook

```
report = tracks.reset_index().sort_values("plays", ascending=False)
report.to_csv(DATA / "tracks-report.csv", index=False)
report.to_csv(DATA / "tracks-report.tsv", sep="\t", index=False)
per_genre = tracks.groupby("genre", as_index=False)["plays"].sum()
with pd.ExcelWriter(DATA / "tracks-report.xlsx", engine="openpyxl") as workbook:
    report.to_excel(workbook, sheet_name="tracks", index=False)
    per_genre.to_excel(workbook, sheet_name="per genre", index=False)
print(pd.ExcelFile(DATA / "tracks-report.xlsx").sheet_names)
```

demo23
writing_files

SpiralTrain developer courses

Plotting

- `.plot(kind=...)` draws a frame or column
- The `Agg` backend writes files without opening a window
- `get_figure().savefig(...)` saves the resulting chart

```
tracks["decade"] = tracks["year"] // 10 * 10
per_decade = tracks.groupby("decade")["plays"].sum()
axes = per_decade.plot(kind="bar", title="Fictional plays by decade")
axes.set_ylabel("plays")
axes.get_figure().tight_layout()
axes.get_figure().savefig(DATA / "tracks-plot.png", dpi=120)
plt.close(axes.get_figure())
print(per_decade)
```

demo24
plot_to_file

SpiralTrain developer courses

Summary : Pandas

- `Series` and `DataFrame` attach labels to data
- Read CSV, Excel and SQL; inspect structure and missing values
- Select by label with `loc` or by position with `iloc`
- Filter with masks; derive columns with array operations
- Summarise groups, join lookup tables and export reports

- Inheritance
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- NumPy
- Pandas
- ▶ **GUI Toolkits**

APPENDIX

- Running Other Programs

- GUI Toolkits
- Installing tkinter
- Root Window and Event Loop
- Widgets
- Widget Values
- Choosing a Genre
- grid Layout
- Events and Callbacks
- ttk.Treeview
- Treeview and SQLite
- Choosing a Toolkit
- GUI Trade-offs

GUI Toolkits

- A graphical interface accepts input through widgets
- An event loop dispatches keyboard, mouse and window events
- Python has a graphical toolkit in the standard library :
  - `tkinter`, with installation depending on the Python distribution
- A playlist form can add tracks and display saved data

SpiralTrain developer courses

Installing tkinter

- **`tkinter` ships with Python, but not with every Python package :**
  - The Windows and macOS installers include it
  - Most Linux distributions split it into a separate package
- **On Debian and Ubuntu it is one command :**

```
sudo apt install python3-tk
```

- **Check it before writing any code :**

```
python -c "import tkinter; print(tkinter.TkVersion)"
```

SpiralTrain developer courses

Root Window and Event Loop

- **A graphical program has a different shape from a script :**
  - A script runs top to bottom and exits
  - A window is created, then waits
- **`Tk()` is the root window, and there is exactly one**
- **`mainloop()` waits for an event, dispatches it, and waits again :**
  - It does not return until the window is destroyed
  - Nothing written after it runs before then

```
root = tk.Tk()
root.title("Playlist")
root.mainloop()
```

demo01
gui_root_window

Widgets

- **Three widgets carry most simple forms :**
  - `Label` shows text, `Entry` takes text, `Button` does something
- **Prefer the themed `ttk` versions over the plain `tk` ones :**
  - Same names, native appearance
- **A widget is created with its parent as the first argument**
- **`command=` takes the function itself, without parentheses**

```
entry = ttk.Entry(root, width=30)
entry.grid(row=0, column=1, padx=8, pady=8)
```

demo02
gui_widgets

Widget Values

- **A widget owns its value, and is asked for it rather than tracked :**
  - `entry.get()` reads what the user typed
  - `label.configure(text=...)` writes it back
- **`configure()` sets any option after construction**
- **The callback is an ordinary function :**
  - It runs when the event happens, not when it is registered

```
def add_track() -> None:
    title = entry.get().strip()
    if not title:
        result.configure(text="Enter a title first")
        return
    playlist.append((title, genre.get()))
    result.configure(text=f"{title}: {genre.get()} ({len(playlist)} tracks)")
    entry.delete(0, tk.END)
```

SpiralTrain developer courses

Choosing a Genre

- `Combobox` combines a text field with a list of choices
- `state="readonly"` limits input to those choices
- `set` selects a value; `get` reads it

```
genre = ttk.Combobox(root, values=["Rock", "Pop", "Soul", "Country", "Electronic"],
                     state="readonly")
genre.set("Rock")
```

SpiralTrain developer courses

grid Layout

- **Widgets do not place themselves, a geometry manager places them**
- **`grid` puts a widget in a row and a column :**
  - `sticky` takes compass points, so `"e"` pins right and `"ew"` fills
  - `padx` and `pady` add space around the widget
  - `columnspan` lets one widget sit across several columns
- **A widget stretches only if its row or column is given a weight :**
  - Weight is a share of the spare space, not a yes or no

```
ttk.Label(form, text=f"{field} :").grid(row=row, column=0, sticky="e", padx=(0, 8), pady=4)
entry = ttk.Entry(form)
entry.grid(row=row, column=1, sticky="ew", pady=4)
entries[field] = entry
```

demo03
gui_grid_layout

SpiralTrain developer courses

Events and Callbacks

- **`bind()` maps an event pattern to a callback :**
  - `<Key>` is any key, `<Key-a>` is one key, `<Button-1>` is a left click
  - `<Configure>` fires when the widget is resized
- **Every bound callback is handed one argument, the event :**
  - It carries `x`, `y`, `keysym` and the widget it happened on
- **A `Button`'s `command=` is the same idea with the event thrown away**

```
root.bind("<Key>", lambda event: report(f"key {event.keysym}", event))
canvas.bind("<Button-1>", lambda event: report("left click", event))
canvas.bind("<Configure>", lambda event: report("resized", event))
```

demo04
gui_events

ttk.Treeview

- `ttk.Treeview` is a real table, with headings and columns
- Rows are inserted, and each one gets an identifier back
- Clicking a heading can re-order the rows :
  - `tree.set` reads one cell, `tree.move` re-places a row
- The widget most internal tools need :
  - Most internal tools are a table of something with a filter above it

```
tree = ttk.Treeview(root, columns=list(COLUMNS), show="headings", selectmode="browse")
for row in rows:
    tree.insert("", tk.END, values=[row[name] for name in COLUMNS], tags=(tag,))
```

demo05
gui_treeview

SpiralTrain developer courses

Treeview and SQLite

- **The same table, filled from SQLite instead of a CSV :**
  - Type a filter, press Return, run the query, refill the table
- **The user's text is a parameter, never formatted into the SQL :**
  - Exactly as in the scraping module

```
sql = f"SELECT {', '.join(COLUMNS)} FROM tracks"
parameters: tuple[str, ...] = ()
if term:
    sql += " WHERE artist LIKE ? OR genre LIKE ? OR title LIKE ?"
    parameters = (f"%{term}%",) * 3
sql += " ORDER BY title"
connection = sqlite3.connect(DB)
found = connection.execute(sql, parameters).fetchall()
connection.close()
return found
```

demo06
gui_treeview_sqlite

Choosing a Toolkit

- `tkinter` :
  - In the standard library
  - Plain, dated, acceptable
  - Small tools and dialogs
- `PySide6` , the Qt binding :
  - A large install
  - Native and modern
  - Real desktop applications
- `Streamlit` :
  - **Opens a browser tab**
  - **No event loop, no callbacks**
  - **Dashboards over data**
- **Model and view :**
  - `tkinter` **holds the rows itself**
  - **Qt asks a model for them**
  - **Which matters at scale**

SpiralTrain developer courses

GUI Trade-offs

- **A graphical program costs more than it looks :**
  - It cannot be run from cron, piped, or driven from a script
  - It needs a display, which rules out most servers
  - It is harder to test than anything else in this course
- **Reach for a window when the user is not you :**
  - And reach for a command-line tool when the user is
- **The command-line tools cover most of the middle ground :**
  - A clear `--help`, real exit codes, and a table printed with `rich`

SpiralTrain developer courses

Summary : GUI Toolkits

- **A GUI inverts the shape of a program :**
  - Build the widgets, then hand control to an event loop
- **`tkinter` is the standard-library binding to Tcl/Tk :**
  - `python3-tk` on Debian and Ubuntu
- **Three widgets and one geometry manager cover most forms :**
  - `Label`, `Entry`, `Button`, placed with `grid`
- **`ttk.Treeview` is the widget most internal tools actually need**
- **Qt and Streamlit are the other two answers, and both are in `reference/gui/`**

- Inheritance
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- NumPy
- Pandas
- GUI Toolkits

APPENDIX

- ▶ **Running Other Programs**

- Running Other Programs
- subprocess.run
- Capturing Output
- Exit Codes
- Argument Lists
- Standard Input and Pipelines
- Timeouts
- Environment and Working Directory
- Missing Programs
- Popen and Streaming Output
- pathlib Alternatives

SpiralTrain developer courses

Running Other Programs

- **Some work belongs to a program that already exists :**
  - `git` , `systemctl` , `rsync` , a vendor's command-line tool
- **`subprocess` starts it, waits for it, and reports back**
- **This is the module for the script that has outgrown its shell :**
  - The logic got complicated, the quoting got worse
  - The commands themselves were never the problem
- **Everything here is standard library, so there is nothing to install**

SpiralTrain developer courses

subprocess.run

- `run()` starts the program, waits, and returns a `CompletedProcess`
- The command is a list : the program, then each argument separately
- Nothing is captured unless you ask, so output goes straight to the terminal

```
finished = subprocess.run(["git", "--version"], check=False)

print()
print("type       :", type(finished).__name__)
print("args       :", finished.args)
print("returncode :", finished.returncode)
print("stdout     :", finished.stdout)
```

demo01
running_a_command

SpiralTrain developer courses

Capturing Output

- `capture_output=True` fills in `stdout` and `stderr`
- `text=True` makes them strings rather than bytes :
  - Without it you get `b'18\n'`, which is almost never wanted
- **After that it is ordinary Python, not a shell pipeline**

```
finished = subprocess.run(
    ["wc", "-l", str(DATA / "listening.log")],
    capture_output=True,
    text=True,
    check=True,
)

print("stdout repr :", repr(finished.stdout))
print("stripped    :", finished.stdout.strip())
print("stderr      :", repr(finished.stderr))
```

demo02
capturing_output

SpiralTrain developer courses

Exit Codes

- **A non-zero exit is information, not always a failure :**
  - `grep` exits 1 when it did not match
- **`check=False` and read `returncode` yourself when the code is meaningful**
- **`check=True` raises `CalledProcessError` when it is not**

```
for pattern in ("ERROR", "NOTHING-MATCHES-THIS"):
    found = subprocess.run(
        ["grep", "-c", pattern, str(LOG)], capture_output=True, text=True, check=False
    )
    print(f"{pattern:<22} returncode={found.returncode}  matched={found.stdout.strip()}")
```

demo03
exit_codes

SpiralTrain developer courses

Argument Lists

- **A list hands the arguments to the program directly :**
  - No shell starts, so nothing splits and nothing expands
- **`shell=True` hands the whole string to a shell first :**
  - A space in a filename becomes two arguments
  - Punctuation becomes syntax

```
listed = subprocess.run(
    ["wc", "-l", str(awkward)], capture_output=True, text=True, check=False
)
print(f"as a list  : rc={listed.returncode}  {listed.stdout.strip()}")
```

demo04
a_list_not_a_string

SpiralTrain developer courses

Standard Input and Pipelines

- `input=` writes to the child's stdin and closes it
- Two commands join without a shell :
  - Run the first, capture it, hand the text to the second
- Clearer than a pipeline string, with nothing for a shell to misread

```
tallied = subprocess.run(
    ["sort"], input=levels.stdout, capture_output=True, text=True, check=True
)
summary = subprocess.run(
    ["uniq", "-c"], input=tallied.stdout, capture_output=True, text=True, check=True
)
```

demo05
input_and_pipes

SpiralTrain developer courses

Timeouts

- **A child that hangs will hang the parent for ever**
- **`timeout=` kills it and raises `TimeoutExpired` :**
  - Nothing is left running behind you
- **Any command touching a network, a device or another machine wants one**

```
try:
    subprocess.run(slow, timeout=1.5, check=True)
except subprocess.TimeoutExpired as error:
    print("raised   :", type(error).__name__)
    print("command  : a sleep of 30 seconds")
    print("waited   :", error.timeout, "seconds")
    print("running behind you.")
```

demo06
timeouts

SpiralTrain developer courses

Environment and Working Directory

- **The child inherits this process's environment by default**
- **`env=` replaces it wholesale :**
  - Copy `os.environ` and add to the copy, or the child loses `PATH`
- **`cwd=` runs the command elsewhere without moving yourself :**
  - Better than `os.chdir()`, which changes it for everything

```
environment = os.environ.copy()
environment["GREETING"] = "set for the child only"
subprocess.run([sys.executable, "-c", CHILD], env=environment, check=True)
```

demo07
environment

SpiralTrain developer courses

Missing Programs

- A missing command raises `FileNotFoundError`, not a non-zero exit
- `shutil.which()` answers the question without running anything
- Check dependencies at the start of a script, not at the point of use :
  - Failing immediately with a clear message beats failing halfway through

```
for program in ("git", "grep", "definitely-not-installed"):
    location = shutil.which(program)
    print(f"{program:<26} {location or '-- not on PATH'}")
```

demo08
finding_the_program

SpiralTrain developer courses

Popen and Streaming Output

- `run()` returns nothing until the command has finished
- `Popen` starts the process and hands back control immediately :
  - Iterating `stdout` yields lines as the child flushes them
- Use it for anything long enough that silence looks like a hang

```
with subprocess.Popen(
    [sys.executable, "-c", CHILD],
    stdout=subprocess.PIPE,
    text=True,
    bufsize=1,
) as process:
    # Iterating the pipe yields lines as the child flushes them.
    for line in process.stdout:
        print("  received :", line.rstrip())
```

demo09
streaming_output

SpiralTrain developer courses

pathlib Alternatives

- Starting a process costs time and brings quoting with it
- Python already does most of what the small commands do :

```
ls *.log      ->  Path.glob             basename  ->  Path.name
wc -l         ->  str.splitlines        test -f   ->  Path.exists
grep          ->  a comprehension       find      ->  Path.rglob
cp / mv / rm ->  shutil.copy, Path.rename, Path.unlink
```

- Reach for `subprocess` when the work belongs to another program

SpiralTrain developer courses

Summary : Running Other Programs

- `subprocess.run()` starts a program, waits, and reports back
- Pass a list, not a string, and leave `shell=True` alone
- `capture_output=True` with `text=True` to read what it printed
- `check=True` to raise on failure, `check=False` when the code means something
- `timeout=` on anything that could hang
- `shutil.which()` to fail early with a clear message
- `Popen` when you need the output as it arrives
- And `pathlib` instead, for anything the filesystem already does